

Score-Conditioned SAO-Small: Complete Pipeline Reference

Last updated: 2026-03-17

This document traces every component of the score-conditioned SAO-Small pipeline at the code level, with exact tensor shapes, file paths, and line numbers.

Table of Contents

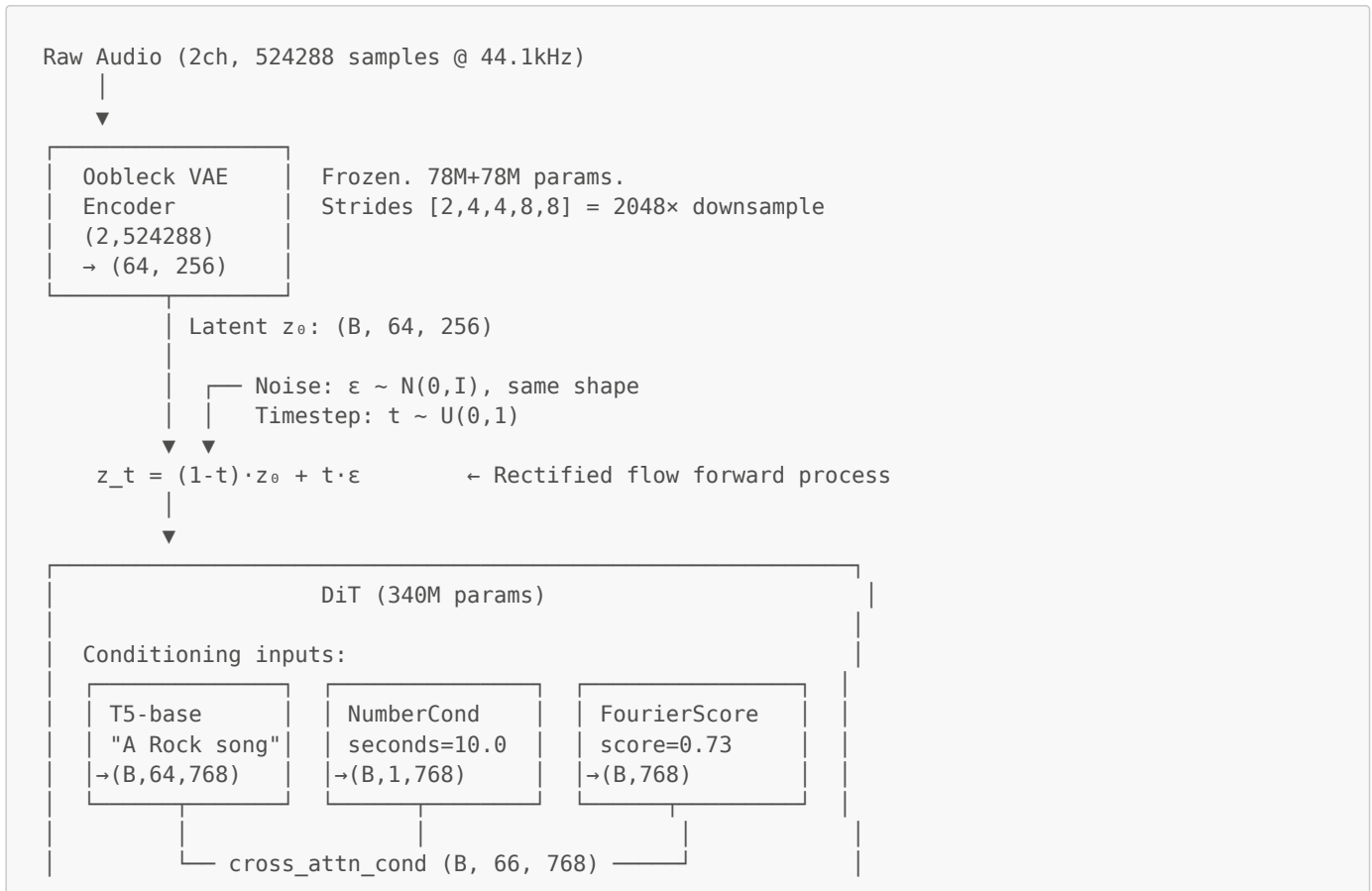
- 1. [System Overview](#)
- 2. [Data Pipeline](#)
- 3. [VAE \(Oobleck Pretransform\)](#)
- 4. [Conditioning System](#)
- 5. [DiT Architecture](#)
- 6. [LoRA Adaptation](#)
- 7. [Noise Schedule & Loss](#)
- 8. [Classifier-Free Guidance](#)
- 9. [Optimizer & Scheduler](#)
- 10. [Freeze/Unfreeze Strategy](#)
- 11. [Validation & Reward Monitoring](#)
- 12. [Experiment History](#)
- 13. [Complete Shape Reference](#)

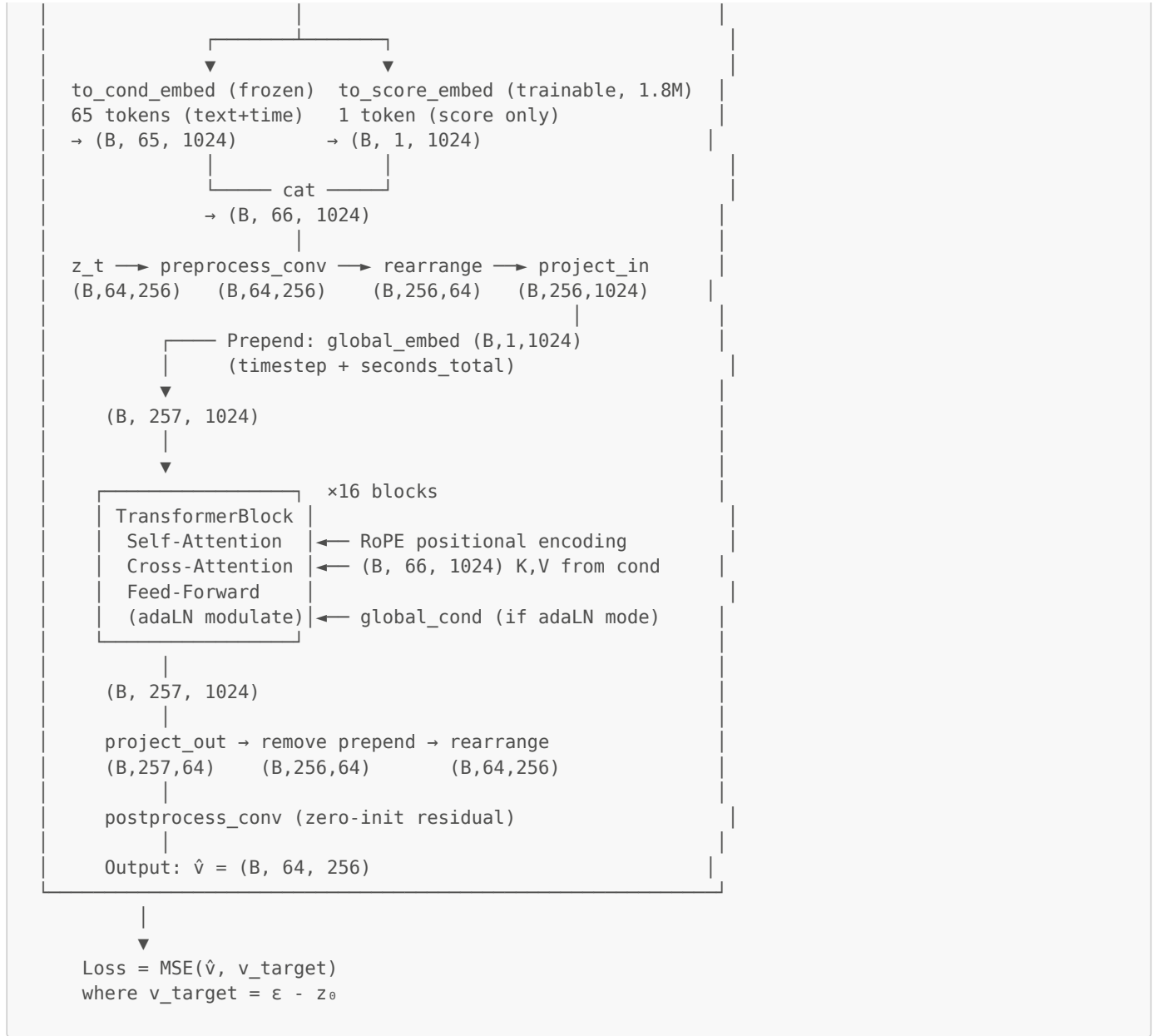
1. System Overview

Goal

Add **quality score conditioning** to a pretrained text-to-audio model (SAO-Small) so that generation quality can be steered at inference time. Higher score → higher quality audio that better matches the text prompt.

Architecture Diagram





Parameter Budget

Component	Parameters	ICME Category
VAE Encoder (Oobleck)	77,989,888	Auxiliary (excluded)
VAE Decoder (Oobleck)	78,122,626	Auxiliary (excluded)
T5-base Text Encoder	~109,000,000	Auxiliary (excluded)
DiT Core	339,063,168	Core
NumberConditioner	198,272	Core (conditioner)
FourierScoreConditioner	788,096	Core (conditioner)
ScoreInputConcatConditioner	9,360	Core (conditioner)
Core total	~340M	500M limit → 160M headroom

2. Data Pipeline

Source: [finetune.py](#) lines 46-124

Dataset Structure

FMA-Large: 106,574 tracks in `/home/yonghyun/fma/data/fma_large/`

Metadata JSONL (`configs/metadata_fma_scored.jsonl`):

```
{
  "reldpath": "/home/yonghyun/fma/data/fma_large/000/000002.mp3",
  "prompt": "A Hip-Hop song.",
  "reward_score": 0.627}

```

FMA Score distribution: range [-7.37, +2.73], median=-0.10, mean=-0.15, std=0.77

MTG-Jamendo: 54,330 tracks in `/home/yonghyun/music-ranknet/data/raw/mtg-jamendo-audio/`

Metadata JSONL (`configs/metadata_jamendo_scored.jsonl`):

```
{
  "reldpath": "/home/yonghyun/music-ranknet/data/raw/mtg-jamendo-audio/46/40146.mp3",
  "prompt": "A slow-paced ambient track...",
  "reward_score": 3.215}

```

Jamendo Score distribution: range [-3.50, +3.21], median=+0.94, mean=+0.91, std=0.51

Sanity check (2026-03-17): Compared metadata scores vs live MusicRankNet(feature) for 20 random FMA tracks — **delta=0.0000 for all 20**. Pre-computed scores are verified correct.

Note on combining datasets: Jamendo scores are ~1.0 higher on average than FMA (commercial music vs user-uploaded). Naively merging risks the model learning dataset identity instead of quality. Score normalization or balanced sampling is required before joint training.

ContinuousScoreDatasetWrapper

File: `finetune.py` lines 46-82

```
class ContinuousScoreDatasetWrapper(Dataset):
    def __getitem__(self, idx):
        audio, metadata = self.dataset[idx]
        metadata = metadata.copy()

        raw_score = metadata.get('reward_score', 0.0)
        # Convert to float (handles tensor/list/scalar)

        # CFG dropout: with probability cfg_drop_rate, replace score with null
        if self.is_training and torch.rand(1).item() < self.cfg_drop_rate:
            score_val = -999.0          # NULL sentinel
        else:
            score_val = float(raw_score)

        metadata['continuous_score'] = score_val
        metadata['score_concat'] = score_val
        return audio, metadata

```

CFG dropout rate: 15% default (`SA_CFG_DROP_RATE`), 30% in current experiments.

Batch Shape

After DataLoader collation with `batch_size=2`:

```
audio:      (2, 2, 524288)    # batch=2, stereo, ~11.9s @ 44.1kHz
metadata: dict {
  'prompt':          list[str] of length 2
  'seconds_total':   tensor(2,)
  'continuous_score': tensor(2,)    # or list of floats
}

```

```
    'reward_score':      tensor(2,)
    'padding_mask':      (2, 524288)
}
```

Filtered Datasets

Config	Tracks	Filter
metadata_fma_scored.jsonl	106,401	All FMA
metadata_fma_scored_top50.jsonl	53,201	FMA score > -0.102 (median)
metadata_fma_scored_top30.jsonl	31,921	FMA score > 0.270
metadata_jamendo_scored.jsonl	54,330	All Jamendo (Qwen2-Audio captions)

3. VAE (Oobleck Pretransform)

Source: Model config `pretransform` section

Frozen during all finetuning. Weights from pretrained SAO-Small.

Encoder

```
Input:  (B, 2, 524288)      # Stereo waveform

Conv layers with strides [2, 4, 4, 8, 8]:
  Layer 1: stride=2  → (B, 128, 262144)
  Layer 2: stride=4  → (B, 256, 65536)
  Layer 3: stride=4  → (B, 512, 16384)
  Layer 4: stride=8  → (B, 1024, 2048)
  Layer 5: stride=8  → (B, 2048, 256)

Bottleneck (VAE): → latent_dim=128 → split → μ, logσ → sample → 64

Output: (B, 64, 256)      # 64 latent channels, 256 timesteps
                        # Compression: 524288/256 = 2048×
```

Decoder

Mirror architecture. (B, 64, 256) → (B, 2, 524288)

4. Conditioning System

Source: `stable_audio_tools/models/conditioners.py`

MultiConditioner Flow

File: `conditioners.py` lines 661-789

```
metadata (dict per sample)
├── 'prompt'      → T5Conditioner → (B, 64, 768)
├── 'seconds_total' ▶ NumberConditioner → (B, 1, 768)
├── 'continuous_score' ▶ FourierScoreCond ▶ (B, 768)
└── 'score_concat' → ScoreInputConcat → (B, 16)
```

T5Conditioner (prompt)

File: `conditioners.py` lines 310-398

```
Input:  ["A Hip-Hop song.", "A Rock song."]      # list[str]

T5-base tokenizer (max_length=64)
  → input_ids: (B, 64)
T5-base encoder (frozen)
  → last_hidden_state: (B, 64, 768)

Output: (B, 64, 768), attention_mask (B, 64)
```

NumberConditioner (seconds_total)

File: `conditioners.py` lines 74-115

```
Input:  [10.0, 10.0]                          # list[float]

Normalize: (val - min_val) / (max_val - min_val)  # [0, 256] → [0, 1]
TimePositionalEmbedding → (B, 768)
  Uses sinusoidal Fourier features, same as timestep encoding
Unsqueeze → (B, 1, 768)

Output: (B, 1, 768), mask (B, 1)
```

FourierScoreConditioner (continuous_score)

File: `conditioners.py` lines 934-982

This is the **core score embedding**. Converts scalar quality score to a rich 768-dimensional representation.

```
Input:  [0.73, -999.0]                        # list[float], -999 = null

Step 1: Convert to tensor
  x: (B, 1)

Step 2: Fourier features
  weight: (128, 1), random init, std=1.0
  f = 2π × x @ weight.T                      # (B, 128)
  fourier_embed = [cos(f), sin(f)]           # (B, 256)

Step 3: MLP mapper
  Linear(256, 768) → SiLU → Linear(768, 768)
  embeds: (B, 768)

Step 4: Null handling
  Where x == -999.0: embeds[null_idx] = 0.0   # Zero vector for null

Output: (B, 768), mask (B, 1)
```

Why Fourier, not Linear(1,768)?

Linear maps score to a single direction in 768d space — score=0.1 and score=0.9 differ only in magnitude, not pattern. Fourier features create **distinct periodic patterns** for different score values, making them much easier for cross-attention to discriminate.

Conditioning Routing

File: `diffusion.py` lines 166-290, `get_conditioning_inputs()`

```
# Config determines which conditioner goes where:
"cross_attention_cond_ids": ["prompt", "seconds_total", "continuous_score"]
"global_cond_ids": ["seconds_total"] # Restored to original SA0

# Cross-attention assembly:
cross_attn_cond = cat([
    prompt_embed,          # (B, 64, 768)
    seconds_total_embed,   # (B, 1, 768)
    score_embed,           # (B, 1, 768) ← unsqueezed from (B, 768)
], dim=1)
# Result: (B, 66, 768)

# Global conditioning:
global_cond = seconds_total_embed # (B, 768)
# → Enters DiT via prepend pathway (self-attention token)
```

5. DiT Architecture

Source: [stable_audio_tools/models/dit.py](#)

DiffusionTransformer Config

```
{
    "io_channels": 64,
    "embed_dim": 1024,
    "depth": 16,
    "num_heads": 8,
    "cond_token_dim": 768,
    "score_cond_num_tokens": 1,
    "global_cond_dim": 768,
    "global_cond_type": "prepend"
}
```

Forward Pass: `_forward()` (dit.py lines 160-302)

Step 1: Cross-Attention Projection (Separate Score Path)

```
# dit.py line 176-195
# Split cross_attn_cond into text/time tokens and score token
n_score = self.score_cond_num_tokens # = 1
main_tokens = cond_input[:, :-n_score, :] # (B, 65, 768) – text + time
score_tokens = cond_input[:, -n_score:, :] # (B, 1, 768) – score only

# Text/time: frozen pretrained projection (no degradation)
main_proj = self.to_cond_embed(main_tokens) # (B, 65, 1024)

# Score: dedicated trainable projection
score_proj = self.to_score_embed(score_tokens) # (B, 1, 1024)

# Rejoin
cross_attn_cond = cat([main_proj, score_proj], dim=1) # (B, 66, 1024)
```

This is the key architectural change: text/time tokens and the score token are projected **independently**. No cross-contamination.

Step 2: Global Conditioning Projection

```
# dit.py line 182-184
global_embed = self.to_global_embed(global_embed)
# to_global_embed = Sequential(Linear(768, 1024, bias=False), SiLU, Linear(1024, 1024,
bias=False))
# FROZEN. Pretrained for seconds_total.

# Shape: (B, 768) → (B, 1024)
```

Step 3: Timestep Embedding

```
# dit.py line 241
timestep_embed = self.to_timestep_embed(self.timestep_features(t[:, None]))
# timestep_features: FourierFeatures(1, 256)
# to_timestep_embed: Sequential(Linear(256, 1024), SiLU, Linear(1024, 1024))

# t: (B,) → t[:, None]: (B, 1) → Fourier: (B, 256) → MLP: (B, 1024)

# Add to global:
global_embed = global_embed + timestep_embed
# Shape: (B, 1024) = seconds_total_proj + timestep_proj
```

Step 4: Input Processing

```
# dit.py line 266-268
x = self.preprocess_conv(x) + x # Residual conv, zero-init at start
x = rearrange(x, "b c t -> b t c") # (B, 64, 256) → (B, 256, 64)
```

Step 5: Prepend Global Conditioning

```
# dit.py lines 254-264
# global_cond_type == "prepend":
prepend_inputs = global_embed.unsqueeze(1) # (B, 1024) → (B, 1, 1024)

# In ContinuousTransformer (transformer.py line 815-823):
x = self.project_in(x) # Linear(64, 1024): (B, 256, 64) → (B, 256, 1024)
x = cat([prepend_inputs, x], dim=1) # (B, 1+256, 1024) = (B, 257, 1024)
```

Step 6: Transformer Blocks (×16)

Each `TransformerBlock` (`transformer.py` lines 582-713):

Input: x (B, 257, 1024), context (B, 66, 1024)

```
┌── Self-Attention ──┐
| Q, K, V = to_q(x), to_k(x), to_v(x) |
| Shape: (B, 257, 1024) → (B, 8, 257, 128) |
| 8 heads, head_dim=128 |
| RoPE positional encoding applied to Q, K |
| Attention: softmax(QK^T/√128) × V |
| Output: (B, 257, 1024) |
└──────────────────┘
      |
      | + residual
      ▼
┌── Cross-Attention ─┐
| Q = to_q(x)         → (B, 8, 257, 128) |
| K = to_k(context)   → (B, 8, 66, 128)  |
| V = to_v(context)   → (B, 8, 66, 128)  |
└──────────────────┘
```

```

Attention: softmax(QK^T/√128) × V
Each of 257 latent positions attends to
66 conditioning tokens:
[text_1..text_64, seconds, score]

Score token influence:
attention_weight ≈ softmax(...)[..., 65]
Competes with 64 text + 1 time token
Output: (B, 257, 1024)

```

| + residual



```

Feed-Forward
Linear(1024, 4096) → GELU → Linear(4096, 1024)
Output: (B, 257, 1024)

```

| + residual



Output: (B, 257, 1024)

When `global_cond_type == "adaLN"` (not used in current best config):

```

adaLN modulation before self-attn and FF:
scale, shift, gate = (to_scale_shift_gate + global_cond_embedder(global_embed)).chunk(6)
x = LayerNorm(x) * (1 + scale) + shift
x = Attention(x) * sigmoid(1 - gate)

```

Step 7: Output Processing

```

# transformer.py line 860
x = self.project_out(x) # Linear(1024, 64): (B, 257, 64)

# dit.py lines 292-302
output = rearrange(x, "b t c -> b c t") # (B, 64, 257)
output = output[:, :, prepend_length:] # Remove prepend: (B, 64, 256)
output = self.postprocess_conv(output) + output # Zero-init residual
# Final: (B, 64, 256)

```

6. Score Projection Strategy

Source: `dit.py` lines 86-100

Current Best: Separate Score Projection (2026-03-17)

```

# dit.py: model construction
# Shared projection for text/time tokens (FROZEN):
self.to_cond_embed = Sequential(Linear(768, 1024), SiLU, Linear(1024, 1024))

# Dedicated projection for score token (TRAINABLE, 1.8M params):
self.to_score_embed = Sequential(Linear(768, 1024), SiLU, Linear(1024, 1024))

```

Forward

Input: cross_attn_cond (B, 66, 768)





Why Separate Projection

Approach	Trainable	Text degradation	Audio quality	Best Corr
Full unfreeze to_cond_embed	1.8M	YES (some genres → silent)	Mostly OK	0.357
LoRA rank=8 on to_cond_embed	802K	Global noise (flatness 0.21)	Poor	0.425
Separate to_score_embed	2.6M	None (frozen text path)	TBD	Running

The fundamental problem with both LoRA and full unfreeze: `to_cond_embed` is shared across all 66 tokens. Any modification for the 1 score token inevitably contaminates the 65 text/time tokens.

Separate projection eliminates this:

- **Text/time:** frozen `to_cond_embed` → zero degradation, pretrained quality
- **Score:** dedicated `to_score_embed` → full expressivity, no interference

Zero-Init Principle

```
to_score_embed.0 (Linear 768→1024): small random init (gradient flow)
to_score_embed.1 (SiLU): activation function
to_score_embed.2 (Linear 1024→1024): ZERO init (starts with zero effect)
```

At initialization, `to_score_embed.2.weight = 0` → output is always zero. The model starts with **exact pretrained behavior** (score token contributes nothing to cross-attention). This is the ControlNet zero-convolution pattern.

Previous: LoRA Adaptation (deprecated)

LoRA applied a rank-constrained residual to the **shared** projection: `output = to_cond_embed(x) + lora_B(lora_A(x))`. This achieved Corr=0.425 but caused global noise degradation (spectral flatness 0.21 vs 0.10 baseline) because the LoRA residual affected all 66 tokens uniformly.

7. Noise Schedule & Loss

Source: `training/diffusion.py` lines 395-477

Rectified Flow

```
# Timestep sampling
t = uniform_sample(0, 1) # (B,)

# Forward process
alphas = 1 - t # (B, 1, 1) after unsqueeze
sigmas = t # (B, 1, 1)
noise = randn_like(diffusion_input) # (B, 64, 256)
noised_inputs = z_0 * alphas + noise * sigmas
# z_t = (1-t)·z_0 + t·ε
```

```
# Target: velocity
targets = noise - diffusion_input          #  $v = \epsilon - z_0$ 
```

Loss

```
# MSE loss
output = model(noised_inputs, t, cond=conditioning) # (B, 64, 256)
loss = MSE(output, targets)                        # scalar

# Score-weighted loss (optional, SA_SCORE_WEIGHTED_LOSS=1)
weight = (reward_score - score_min) / (score_max - score_min)
weight = weight.clamp(0.1, 1.0)                  # Never zero out any sample
loss = loss * weight.mean()                      # Upweight high-quality samples
```

Score-weighted loss is **orthogonal** to score conditioning — it changes the loss magnitude based on quality, not the model architecture.

8. Classifier-Free Guidance (CFG)

Training-Time Dropout

File: `finetune.py` lines 77-78

```
# With probability cfg_drop_rate (15% or 30%):
score_val = -999.0 # NULL sentinel

# In FourierScoreConditioner:
if score == -999.0:
    embedding = zeros(768) # Null = zero vector
```

This teaches the model two modes:

- **Conditioned** (85-70% of training): knows the score, generates accordingly
- **Unconditioned** (15-30%): generates without score information

Inference-Time CFG

File: `dit.py` lines 413-521

```
# Batch construction: [conditioned, unconditioned]
batch_inputs = cat([x, x], dim=0)          # (2B, 64, 256)

# Cross-attention:
batch_cond = cat([
    cross_attn_cond,                # (B, 66, 768) with real score
    zeros_like(cross_attn_cond)     # (B, 66, 768) all zeros
], dim=0)                               # (2B, 66, 768)

# Global embed:
batch_global = cat([
    global_embed,                   # (B, 768) with real seconds
    zeros_like(global_embed)       # (B, 768) zeros
], dim=0)                             # (2B, 768)

# Forward pass produces (2B, 64, 256)
cond_output, uncond_output = chunk(output, 2)

# CFG formula:
final = uncond_output + cfg_scale * (cond_output - uncond_output)
```

cfg_scale = 3.5 (default). Higher values amplify conditioning more aggressively. The score signal is amplified alongside the text signal.

Critical fix applied: Previously, the unconditioned pass used the **same** global_embed as the conditioned pass (`cat([embed, embed])`), making the score invisible to CFG. Fixed to use zeros for uncond pass.

9. Optimizer & Scheduler

Source: `finetune.py` lines 206-233

```
optimizer = AdamW(
    trainable_params,
    lr=5e-5,           # SA_LR (or 1e-4 for LoRA experiments)
    weight_decay=1e-3  # SA_WEIGHT_DECAY
)

scheduler = CosineWithHardRestarts(
    warmup_steps=1000,      # SA_WARMUP_STEPS
    total_steps=300000,     # SA_TOTAL_STEPS
    num_cycles=18           # SA_NUM_CYCLES
)
```

Schedule Visualization



Effective Batch Size

Per-GPU batch:	2
Gradient accum:	4
GPUs:	2
Effective batch:	$2 \times 4 \times 2 = 16$

10. Freeze/Unfreeze Strategy

Source: `finetune.py` lines 22-39, 142-203

Profiles

Profile	Unfrozen Parameters	Trainable Count
<code>minimal</code>	<code>continuous_score, score_bin</code>	~788K
<code>lora_xattn</code>	<code>continuous_score, score_bin, cond_embed_lora</code>	~802K
<code>xattn</code>	<code>continuous_score, score_bin, to_cond_embed</code>	~1.8M
<code>separate_score_proj</code>	<code>continuous_score, score_bin, to_score_embed</code>	~2.6M

Profile	Unfrozen Parameters	Trainable Count
adaln	continuous_score, score_bin, to_global_embed, to_scale_shift_gate, global_cond_embedder	~9.3M

Current: `separate_score_proj`

Unfrozen:

conditioner.conditioners.continuous_score.*

788,096 params

├─ fourier_features.weight

(128, 1)

├─ mapper.0.weight

(768, 256)

├─ mapper.0.bias

(768,)

├─ mapper.2.weight

(768, 768)

└─ mapper.2.bias

(768,)

model.model.to_score_embed.0.weight

(1024, 768) = 786,432

model.model.to_score_embed.2.weight

(1024, 1024) = 1,048,576

Total trainable:

~2.6M / 497M (0.52%)

Frozen:

Everything else (494.4M params):

VAE encoder/decoder, T5, DiT transformer blocks,

to_cond_embed, to_global_embed, to_timestep_embed,

all self-attention, all cross-attention, all FFN

Weight Initialization for New Parameters

File: `finetune.py` lines 142-187

```
# ControlNet zero-init pattern:
# - Output layers: zero (no effect at start)
# - Intermediate layers: small random (gradient can flow)

KEEP_RANDOM = ["continuous_score", "score_concat", "cond_embed_lora_A",
               "global_cond_embedder.0", "to_score_embed.0"]

for name, param in model.named_parameters():
    if name not in pretrained_keys:
        if any(pat in name for pat in KEEP_RANDOM):
            param.data.normal_(0, 0.02) # Small random
        else:
            param.zero_() # Zero-init
```

copy_state_dict Zero-Padding

File: `models/utils.py` lines 6-38

When `input_concat` adds channels (64 → 80), the pretrained `preprocess_conv` weight (shape 64×64×3) is zero-padded to (80×80×3). New channels start with zero effect.

11. Validation & Reward Monitoring

Source: `training/reward_monitor.py` lines 158-338

Validation Flow

Every 5000 steps:

for 100 samples (10 per bin × 10 bins):

```
1. Pick prompt from val set
2. Set target_score from bin threshold
3. Generate audio with CFG:
  - Positive: prompt + score=target
  - Negative: "" + score=-999 (null)
  - cfg_scale=3.5, steps=50
4. Extract features from generated audio:
  - MERT:      (1, 1024)   @ 24kHz
  - CLAP audio: (1, 512)   @ 48kHz
  - CLAP text: (1, 512)   from prompt
  - Flag:      (1, 1)     = 1.0
5. Concatenate: [flag, clap_audio, mert, clap_text] = (1, 2049)
6. Score = MusicRankNet(features)
7. Log target vs measured score

Compute:
  Pearson correlation(targets, measured)
  Pairwise monotonicity
  Per-bin average scores
```

Music-RankNet Architecture

File: [music-ranknet/src/model.py](#)

```
Input: (1, 2049)
      [flag(1), CLAP_audio(512), MERT(1024), CLAP_text(512)]

Linear(2049, 1024) → BatchNorm → ReLU → Dropout(0.3)
Linear(1024, 512)  → BatchNorm → ReLU → Dropout(0.3)
Linear(512, 256)   → BatchNorm → ReLU → Dropout(0.3)
Linear(256, 128)   → ReLU
Linear(128, 1)     → Score output (scalar)
```

Target Score Bins

10 bins from validation thresholds (percentiles of FMA score distribution):

Bin	Target Score	Meaning
Top 10%	0.91	Highest quality
Top 20%	0.60	
Top 30%	0.38	
Top 40%	0.16	
Top 50%	-0.02	Median
Top 60%	-0.18	
Top 70%	-0.34	
Top 80%	-0.52	
Top 90%	-0.78	
Top 100%	-1.52	Lowest quality

12. Experiment History

Timeline

Version	Config	Pathway	Trainable	Result	Root Cause
---------	--------	---------	-----------	--------	------------

Version	Config	Pathway	Trainable	Result	Root Cause
v1-v4	prepend	global (prepend)	1.5K	Corr~0	adaln profile with prepend config = no-op
v5	adaln_pure	global (adaLN)	7.4M	Corr=-0.03	sum-merge + dual path
v6	adaln_pure	adaLN only	7.4M	Corr~0	global_cond_embedder dead gradient
v7	adaln_pure	adaLN + random init	9.3M	Corr~0	Score redundant for denoising
v8	adaln_pure	adaLN + full random	9.3M	NaN	fp16 overflow from large adaLN
v9	adaln_pure	adaLN + to_global_embed	9.3M	Corr~0	Same fundamental issue
v10	adaln_pure	adaLN + CFG fix	9.3M	Corr=-0.04	adaLN unsuitable for weak signal
xattn v1-2	xattn	cross-attention	1.8M	Corr=0.357	Works! But some genres → silent
Fourier+xattn	xattn	Fourier + cross-attn	1.8M	Corr=0.342	Text degrades (shared proj)
LoRA v2 (r=8)	lora_xattn	Fourier + LoRA	802K	Corr=0.425	High Corr but global noise (flatness 0.21)
LoRA+weighted+top50	lora_xattn	LoRA + score-weighted	802K	Corr=0.325	Stopped: weighted loss hurts
LoRA r=32	lora_xattn	Fourier + LoRA	845K	—	Stopped: same noise problem expected
Separate proj v1	separate_score_proj	Fourier + dedicated proj	2.6M	Running	No cross-contamination

Key Findings

- **adaLN fails for score conditioning:** 6 attempts (v5-v10), Corr≈0. Root cause: score enters global_cond which is sum-merged with seconds_total — model cannot disentangle the two signals.
- **Cross-attention works** (Corr=0.357) but unfreezing **to_cond_embed** destroys text generation (Classical/Blues/Lo-Fi → near-silent).
- **LoRA on to_cond_embed:** Corr=0.425 but **global audio quality degradation**. Spectral flatness rose from 0.10 (pretrained) to 0.21 (epoch 8). LoRA residual is applied uniformly to all 66 tokens, contaminating text projection. Stopped.
- **Separate score projection** (current): Dedicated **to_score_embed** for score token only. **to_cond_embed** stays frozen → zero text degradation. 2.6M trainable params, architecturally cleanest solution.
- **Score-weighted loss hurts:** peaked at 0.325, lower than vanilla LoRA.
- **Score in both global + cross-attn:** Not viable (sum-merge problem).
- **Demo config bug found:** **continuous_score: 10.0** was out-of-distribution (FMA range: [-7.37, +2.73]). Fixed to use real percentile values.

Current Runs (2026-03-17)

GPU	Experiment	Status
0	FMA caption generation (Qwen2-Audio)	84K/107K (79%)
3-7	Reserved / Do not use	—
8-9	Separate proj v1 (case3b_separate_score_proj_v1)	Just started

Data Readiness

Dataset	Tracks	Captions	Scores	Status
FMA-Large	106,574	Qwen2-Audio (84K/107K)	All scored	Training active
MTG-Jamendo	54,330	Qwen2-Audio (all done)	All scored (2026-03-17)	Ready, pending normalization

13. Complete Shape Reference

Forward Pass: Raw Audio → Loss

Stage	Variable	Shape	Notes
DataLoader	audio	(2, 2, 524288)	Stereo, ~11.9s
	metadata['continuous_score']	(2,)	Float, -999=null
	metadata['reward_score']	(2,)	For weighted loss
VAE Encode	diffusion_input	(2, 64, 256)	2048× compressed
Conditioner	T5 prompt embed	(2, 64, 768)	64 text tokens
	seconds_total embed	(2, 1, 768)	1 time token
	score embed	(2, 768)	Fourier + MLP
Routing	cross_attn_cond	(2, 66, 768)	All tokens concat
	global_cond	(2, 768)	seconds_total
Noise	t	(2,)	Uniform [0,1]
	noise ϵ	(2, 64, 256)	$N(0,I)$
	noised z_t	(2, 64, 256)	$(1-t)z_0 + t\epsilon$
	target v	(2, 64, 256)	$\epsilon - z_0$
DiT: Projection	cross_attn_cond	(2, 66, 1024)	to_cond_embed(65) + to_score_embed(1)
	global_embed	(2, 1024)	to_global_embed(seconds)
	timestep_embed	(2, 1024)	Fourier + MLP
	global_embed	(2, 1024)	global + timestep sum
DiT: Input	x after preprocess	(2, 64, 256)	Zero-init residual conv
	x rearranged	(2, 256, 64)	Channels last
	x after project_in	(2, 256, 1024)	Linear(64, 1024)
	prepend_embed	(2, 1, 1024)	Global embed unsqueezed
	x with prepend	(2, 257, 1024)	Concat along seq
DiT: Transformer	Self-attn Q,K,V	(2, 8, 257, 128)	8 heads
	Cross-attn Q	(2, 8, 257, 128)	From latent
	Cross-attn K,V	(2, 8, 66, 128)	From conditioning
	After 16 blocks	(2, 257, 1024)	
DiT: Output	project_out	(2, 257, 64)	Linear(1024, 64)
	remove prepend	(2, 256, 64)	Drop first token
	rearrange	(2, 64, 256)	Channels first
	postprocess	(2, 64, 256)	Zero-init residual

Stage	Variable	Shape	Notes
Loss	output $\hat{v}$	(2, 64, 256)	Model prediction
	MSE($\hat{v}$, v)	scalar	Loss value

Inference: Prompt + Score → Audio

Stage	Shape	Notes
Input prompt	str	"A Rock song."
Input score	float	0.91
T5 encode	(1, 64, 768)	
Score embed	(1, 768)	
CFG doubling	(2, 64, 256)	[cond, uncond]
50 denoising steps	(2, 64, 256) → ...	Euler/DDIM solver
CFG combine	(1, 64, 256)	uncond + scale*(cond-uncond)
VAE decode	(1, 2, 441000)	10s @ 44.1kHz stereo
Output	WAV file	10s, 44.1kHz, stereo

This document is part of the SAO Score Conditioning project for ISMIR 2026 and ICME Challenge.